June 20, 1955
R. D. Richtmyer

TRIAL BALLOON NO. n+1

On the supposition that no machine code yet devised is just what we want, it would seem reasonable for people to experiment a little bit, now and then, with devising further codes. I toss out here some ideas for a two-address code which may have some virtues. All the ideas (with possible exception of one) are stolen in obvious ways from existing machines (SEAC, MANIAC, UNIVAC, NORC, LARC).

The idea is to provide a code which will have the simplest possible vocabulary, from the point of view of the programmer (just over a dozen instructions, each represented by a mnemonic symbol), and yet have all the flexibility known in the art today. I attempt to achieve this by being somewhat extravagant with bits in the instructions -- there are quite a few characters which would often be left blank and, therefore, unityped as zeros -- and also perhaps by being a little bit callous concerning the poor engineers who would have to design the circuits.

I am thinking of a machine of the general class patterned after the prescriptions of Burks, Boldstine and Von Neumann (Princeton, 1946 -- I think of this class as including all the machines mentioned above, except the 4-address version of the SEAC, which had no control counter). Basic units would be:

Memory ($\approx$ 64,000 words)

Control Counter (1)

Control Register (1)

Arithmetic Registers (2, addressable)

B Registers (15)

Function table matrix and control circuitry

Special Registers (around 10, addressable, for use in abnormal conditions like overflows)

Supervisory control keyboard and printer

Input-Output (simple)

The justification for being lavish with use of bits in constructing the vocabulary is that I am thinking of problems, like multidimensional reactor and fluids dynamics problems, in which a very large internal memory is required for numerical storage while the coding itself is not likely to exceed a few thousand words.

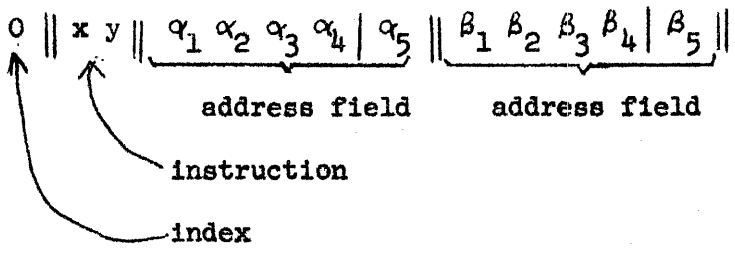
Word Structure. A word consists of a 3-or 4-bit index followed by a 52-bit main part. The index determines the interpretation of the main part as follows:

Index	Main Part
+	magnitude of number ≥ 0
-	magnitude of number < 0
0	instruction
m	meta instruction
α	alphabetic
f^+	floating point representation of number ≥ 0
f^-	floating point representation of number < 0

(The last two were added at the suggestion of H. Goertzel, and would be used if floating point operations are wired into the machine.) Keyboard and printers must be prepared to operate in the appropriate manner, depending on the index of the word being typed or printed. Numbers and addresses are binary and written in hexadecimal notation (except when converted to decimal) with digits 0, 1, ..., F of 4 bits each. The digits are also included as special cases of alphabetic characters (presumably by leaving the first 2 bits of the 6 both zero). By use of suitable type fonts one distinguishes the digits A, B, C, D, E, F from the letters a, b, c, d, e, f, as well as the digits 0, 1 from the letters o, i, ℓ.

Fixed point numbers may be thought of as lying in the range $-1 < x < 1$ or as integers in the range $-2^{52} < x < 2^{52}$. The first of these points of view will be adopted in the following descriptions.

A fixed point number consists of a sign (the index) followed by 13 digits. An instruction consists of the index 0 (which may be left blank in coding) followed by two alphabetic characters to specify the instruction, followed by 10 digits arranged in two address fields, α and β, of 5 digits each, viz.



The first four digits in either address field specify a memory location (address) and the fifth digit, if not zero, specifies which of the 15 B registers is to be used in modifying the address.

The addresses of the two main registers are 0000 and 0001 for which I will write simply "0" and "1." These registers are interchangeable with each other and with the memory locations 0002 through FFFF in the sense that the content of either register may be used as either operand (or both) for any arithmetic operation.

Most instructions have 0 (zero) for the second character. When only one character is written, it is understood that the second one is zero.

I denote by (α) the word stored at location α.

In some cases the result of an operation is the 104 bit number whose more significant part is in register 0 and less significant part in register 1.

Vocabulary. In the following descriptions of the basic orders, I underline the word with which the mnemonic character of the instruction symbol is supposed to be associated.

a Add (α) and (β). The sum appears in register 1 with carry if any, in register 0. Both registers have the sign of the sum.

(Either address, or both, may, of course, refer to either of the registers 0 or 1.)

s Subtract: $(\alpha) - (\beta)$. Similar to above

m Multiply $(\alpha) \times (\beta)$. Similar to above. The 104 bit product appears in registers 0 and 1.

mr Multiply and round. First multiply as above. Then, if the magnitude of (1) is $\geq 1/2$, it is replaced by its complement, its sign is changed, and the magnitude of (0) is increased by 1 in the last place.
(Whether rounding takes place or not, the full product is exactly $(0) + 2^{-52}(1)$).

d Divide (α) by (β). Quotient appears in register 0, remainder in register 1. The exact quotient is

$$\frac{(\alpha)}{(\beta)} = (0) + 2^{-52}\,\frac{(1)}{\text{denominator}}\,.$$

(Note that, in contrast to pattern of most machines, the result of the previous instruction may be used directly for either numerator or denominator).

dr Divide and Round. The rounding is similar to that for multiplication in that we now allow for a remainder having different sign from the quotient, but so chosen that (0) is as close as possible to the true quotient, and the above equation for $(\alpha)/(\beta)$ remains valid.

di Special division for calculation with integers. Same as "d", except that the numerator, instead of being just (α) is the 104-bit number

$$(\alpha) + 2^{-52}\,(1).$$

Generally register 1 must be loaded up before instruction di is performed.

(This instruction would probably be little used, and I wouldn't insist on it. I include it here because in a sense it is the true inverse of the m instruction).

l Shift the number in location α to the left 8000-β (binary) places, leaving the sign unaltered, and leaving the main registers unaffected unless α is either 0 or 1.

(If $\beta < 8000$ this becomes a right shift. Amount of shift can be modified by a B register or by modifying the instruction itself).

lw Ignore α, shift the 104-bit number in 0 and 1 to the left as a whole 8000-β places, leaving both signs unaltered.

00 ("zero") (left blank in writing) Put (α) in location β, leaving (α) unaltered in α.

(The blank instruction field is mnemonically appropriate since no operation is performed on the number).

(UNIVAC equivalents: B, H, L, I, K, LI, BH, IB, JB, skip)

(A completely blank word becomes a skip instruction)

Transfers of control are "transfer on plus," "transfer on zero," and "transfer unconditionally", designated p, z, u.

p If (α) $\geq$ 0, transfer control to location β

z If (α) = 0, transfer control to location β

u Transfer control to location β. (In normal use, the α field is left blank). If the α field is $\neq$ 0, first record the present control counter reading in the word at location α, in "β-address position"; i.e. in the first four of the last five hexadecimal digits of word at location α, leaving

remainder of word at location α unaltered; then transfer control to location β. (Similar to UNIVAC U and RU except that in the latter case the exit line of the subroutine is not restricted to be an unconditional transfer instruction).

e — extract. Various combinations are possible here, and I don't know which is best. For example, use $[\beta]$ as extractor, extract from α to register 0 (as in UNIVAC) and have the logical product of $[\alpha]$ and $[\beta]$ appear in register 1 (for those who like logical product). In any case, it should be a binary extraction and the index of the extractor should be unrestricted (then, at least if all 64 possible "alphabetic" characters are usable, the extractor can be written in any form desired: as a number, an instruction, or in alphabet: form).

xn (n variable, = digit or alph. char.) — transmit words to or from external medium number n, from or to memory locations α, $\alpha + 1$, ..., β

For example: n=0, keyboard input; n=1, supervisory control printer; n=2, read tape one block forward; n=3, read tape one block backward (variable block length, but with space between blocks and provision for setting one of the "abnormal condition" toggles if wrong number of words is found in the block); n=4, write on tape; n=5, rewind tape. I assume that with $\approx$64,000 wds. internal memory, tapes would not be much

used in scientific problems except for input, output, and rerun points.

(note that "xo", or just "x" becomes a stop order if we identify n=0 with keyboard input, and is in fact the only stop order needed, if we follow HG's idea that all communication with the machine should be through a keyboard on the supervisory control printer, because every stop must be followed by an input-ready condition if the machine is ever to be used again)

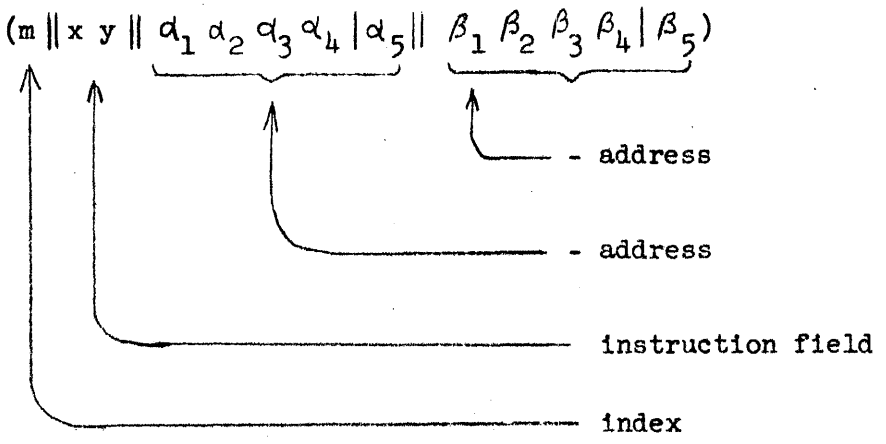
sb Set B register number α_5 (see foregoing layout for word structure) to address $\alpha_1\ \alpha_2\ \alpha_3\alpha_4$ and also B register number β_5 to $\beta_1\ \beta_2\ \beta_3\ \beta_4$.

[Either address field may be left blank, if you wish to set only one B register. Each B register holds just 16 bits. When the contents of a B register are used to modify an address in the code, the addition is done mod 2^{16} (i.e. no final carry to the left). Therefore there is no need for negative quantities in the B register. For example, subtracting 7 from an address is the same as adding FFF9 to it (mod 2^{16}).]

ib Increment B register number α_5 by adding $\alpha_1\ \alpha_2\ \alpha_3\ \alpha_4$ to its contents (mod 2^{16}) and transfer control to location β unless the B register now contains 0000, in which case do not transfer control.

The meta instruction. Further vocabulary can be manufactured by the programmer ad libitum (or ad nauseam) by use of the meta instruction.

This looks like an ordinary instruction except that 1) the index is m, not 0; 2) the combination in the instruction field does not correspond to one of the above, "wired-in" instructions. It serves the purpose of defining further instruction symbols in terms of subroutines. Let the metainstruction be



This has the effect of informing the machine that henceforth, until further notice, whenever an instruction is encountered with "xy" in the instruction field, the machine should execute the subroutine with entrance at $\alpha_1 \alpha_2 \alpha_3 \alpha_4$ and exit at $\beta_1 \beta_2 \beta_3 \beta_4$ (these addresses may be modified by B registers unless $\alpha_5 = \beta_5 = 0$).

This behaves like UNIVAC "RU" except for the following differences:

1) It is not necessary to memorize the entrance and exit addresses of oft-used subroutines; instead one memorizes only the symbol "xy" which may be chosen for its mnemonic qualities (see examples below).

2) If desired, the registers may be loaded up at the same time the excursion to the subroutine is directed.

Specifically, suppose that subsequent to the above meta-instruction, there appears the real instruction :

$$(0 \| x\ y \| \alpha_1' \alpha_2' \alpha_3' \alpha_4' \| \alpha_5' \| \beta_1' \beta_2' \beta_3' \beta_4' \| \beta_5' \|),$$

the machine will put (α') in register 0, (β') in register 1, then execute the subroutine from α to β , then return to previous line of instruction after completing the subroutine.

Examples of instructions that could be generated by use of the meta-instruction are:

sq — take the square root of the complex number $(\alpha) + i(\beta)$; put the real part in register 0 and the imaginary part in register 1.

pr — convert to decimal, edit with decimal points, and print in a column on the high speed printer, the numbers from mem. location α to mem. location β

af, sf, mf, df — floating-point arithmetic instructions.

It is likely that some of the instruction I have omitted from the "wired-in" list, should in fact be wired in rather than generated by use of the meta-instruction to gain speed (this is probably true of the

floating-point instruction). This is however purely an economic question. If an instruction which the programmer tries to generate is, in fact, wired in, nothing is lost - - - the programmer will merely be pleasantly surprised by the speed with which the machine does his problem.

(The amount of hardware required for the meta-instruction could be more or less, depending on the speed desired. For example, let us agree to put the meta-instructions in mem. locations FFFF, FFFE, etc., the most oft used ones first. When the machine, in trying to execute an instruction, finds no function table output, it searches mem. locations FFFF, FFFE, etc. until it finds one whose instruction field xy agrees with that currently in the control register. Then, after fetching (α') and (β') to register 0 and 1 it puts "U α β" in the control register and executes same. This would require some control, gating and coincidence circuits but not additional registers)

Miscellaneous. An alphabetic word (index "$\frac{1}{2}$") has eight characters plus four wasted bits. (Used in editing)

The special registers for abnormal conditions might behave like UNIVAC 000 on overflow, in that when the abnormal condition occurs, the instruction in the corresponding special register is interposed in the normal sequence of instructions.

There should probably be a special register for each of the following conditions:

1) addition-subtraction overflow
2) division overflow
3) left shift overflow
4) wrong number words in block on tape

5) parity failure in tape read

6) trying to interpret a number as an instruction

7) trying to perform arithetical operations on alphabetic characters

8) printer ready

etc.

The memory-clear operation should probably automatically put a stop instruction in each of these special registers.* Since these registers are addressible, the program can alter them later, if desired.

In addition to the ~~sum~~ seven choices of index listed on page 2, there should be one or more additional ones for manual control of the machine when a type in is called for. For example, one index, when typed, might have the effect that the remainder of the typed word goes into the control register where it is executed before the machine continues with the program. Another one might have the effect that the memory is cleared and then the remainder of the typed word goes into the control counter and is executed. Another might set the dial-breakpoint to an address specified somehow by the remainder of the typed word, and another might cause those sense switches (a la 701) to be turned on for which corresponding digits occur in the remainder of the typed word, and the others turned off.

In connection with the last mentioned index I assume there are nine "sense switches" or toggles used in connection with u instructions.

---

*except the one for printer ready, which should receive a skip instruction.

If instead of u0 we write un, where n = 1, 2, ---, 9, the unconditional transfer is inhibited if sense switch number n is turned on.